

◆ Gemini

```

import cv2
import numpy as np
from google.colab.patches import cv2_imshow

image_path_colab = "/content/PICTURE.png"

img = cv2.imread(image_path_colab)

if img is None:
    print(f"Error: Could not load image from {image_path_colab}. Make sure the file exists and is accessible.")
else:
    rows, cols = img.shape[:2]

    # Define source points (corners of the original image)
    src_pts = np.float32([[0, 0], [cols - 1, 0], [cols - 1, rows - 1], [0, rows - 1]])

    # Define destination points for the DLT (e.g., to create a trapezoidal effect)
    # These points can be adjusted to achieve different transformation effects
    dst_pts = np.float32([
        [int(0.2 * cols), 0],          # Top-left corner moved inwards
        [int(0.8 * cols), 0],          # Top-right corner moved inwards
        [cols - 1, rows - 1],         # Bottom-right corner unchanged
        [0, rows - 1]                 # Bottom-left corner unchanged
    ])

    # Compute the homography matrix using DLT algorithm
    # M is the 3x3 transformation matrix
    # mask identifies inliers if RANSAC or other robust methods are used (here it's None)
    M, _ = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

    # Apply the perspective transformation
    dlt_transformed_img = cv2.warpPerspective(img, M, (cols, rows))

    # Save the transformed image
    cv2.imwrite('DLT_Transformed_Image.png', dlt_transformed_img)

```

```
cv2.imwrite('DLT_Transformed_Image.jpg', dlt_transformed_img)  
print("DLT transformation applied and image saved as 'DLT_Transformed_Image.jpg'.")  
  
# Display the transformed image  
print("Displaying the DLT Transformed Image:")  
cv2.imshow('DLT_Transformed_Image', dlt_transformed_img)
```

DLT transformation applied and image saved as 'DLT_Transformed_Image.jpg'.
Displaying the DLT Transformed Image:

